Page No.:
Date: / /

Lecture -15

Data structure and Algorithm

* Search in a rotated Array :

0	1	2	3	4	5	6	7	8
5	6	7	8	9	1	2	3	4

key = 4

By Binary search :

Originally,

	1	2	3	4	5	6	7	8	9
Rotate 1:	9	1	2	3	4	5	6	7	8
Rotate 2:	8	9	1	2	3	4	5	6	7
Rotate 3:	7	8	9	1	2	3	4	5	6

In Rotate Array,

It is Sorted in one direction only.

← or →

*

0	1	2	3	4	5	6	7	8
5	6	7	8	9	1	2	3	4

$$\frac{0+8}{2} = 4$$

arr[4] == key

↳ False.

Teacher's Signature.....

Page No.
Date / /

Check:

arr[start] <= arr[mid]

5 <= 9

↳ true.

↳ Check:

arr[start] <= key <= arr[mid]

↳ True

↳ check in left

↳ False

↳ check in right

A/c to problem

↳ it is false

↳ go to right

*

5	6	7	8
1	2	3	4

$$\frac{5+8}{2} = 6$$

arr[6] == key

↳ False.

Check:

arr[start] <= arr[mid]

↳ true.

Check:

arr[start] <= key <= arr[mid]

↳ false

↳ go to right

Teacher's Signature.....

7 | 8
3 | 4

$$mid = \frac{7+8}{2} = 7$$

arr[mid] == key

↳ False

go to right

8
4

$$mid = \frac{8+8}{2} = 8$$

arr[mid] == key

↳ true.

return mid.

Code:

```
int search (int A[], int l, int h, int key) {
        int index = -1;
        while ( l <= h) {
            int mid = h+ (l-h) / 2;

        if (A[mid] == key) {
                return mid;
        }
```

```
else if ( A[mid] > A[l] ) {
        if ( A[mid] < key || key < A[l] ) {
                l = mid + 1;
        }
        else {
                h = mid - 1;
        }
}
else {
        if ( A[l] <= key || key < A[mid] ) {
                h = mid - 1;
        }
        else {
                l = mid + 1;
        }
    }
}
return index;
}
```

Page No.:
Date: / /

Aggressive Cows

Given : An Array
↳ consist of n integer
Array denote the position of stall.

K : No. of cows.

Task :
Assign stalls to K cows
↳ So the distance b/w two cows are maximum.

Example :

N = 5 K = 3
↓ ↳ No. of cows
No. of stalls

Stall No : 1 2 4 8 9

3 cow ko stall me dalna hai Maximum distance pe taki koi ek dusre se ladai na kare.

1 2 4 8 9 min

$C_1 \xleftarrow{1} C_2 \xleftarrow{2} C_3$ ⇒ (1)

$C_1 \xleftarrow{3} C_2 \xleftarrow{4} C_3$ ⇒ (3)

$C_1 \xleftarrow{3} C_2 \xleftarrow{5} C_3$ ⇒ (3)

Teacher's Signature.....

Max = 3

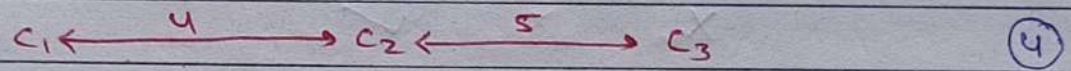
Example 2

$n = 5$ $K = 3$

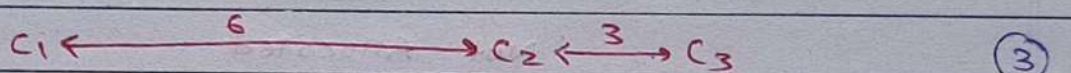
10 1 2 7 5

↳ sort

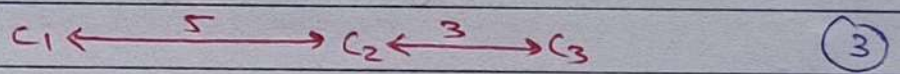
1 2 5 7 10



(4)



(3)



(3)

Maximum = 4

start from taking distance → 1

If answer found (No. of cows allocated)
↳ Move right

If answer Not found (No. of cows not allocated)
↳ Move left

start = 1
end = Max array - Min value.

Teacher's Signature.....



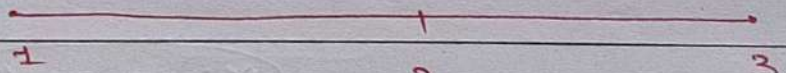
1 4 8

$mid = \frac{1+8}{2} = 4$

1	2	4	8	9
C1	2-1	4-1	8-1	9-8 = 1
	=1	=3	=7	X
	X	X	✓	
			C_2	

Not Allocated

high = Mid - 1



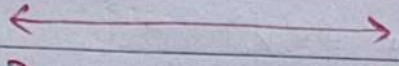
$Mid = \frac{1+3}{2} = 2$

1	2	4	8	9
C1	2-1=1	4-1	8-4	
	X	=3	=4	
		C_2	C_3	

Allocated

answer = 2

low = mid + 1.



mid = 3

1	2	4	8	9
C1	2-1	4-1	8-4	
	= 1	= 3	= 4	
	X	C_2	C_3	

Allocated

Answer = 3

low = mid + 1 = 4



low > high → condition fails.

```
start = 1
end = Max - Min
int ans = -1
while ( start <= end ){
int mid = ( start + end) / 2;
    int Count = 1;
int last_pos = arr[0];

for(int i = 0; i<n; i++ ){
    if ( arr[i] - last_pos >= mid){
        Count ++ ;
        last_position = arr[i];
}
```

Teacher's Signature.....

```
            if (count == k) {
                break;
            }
        }
        if (count == k) {
            ans = mid;
            start = mid + 1;
        }
        else {
            end = mid - 1;
        }
    }
    return ans;
}
```

Teacher's Signature.....